



New issue



[Performance] [Web] Using the onnxruntime-web package (wasm backend) with Node.js is 1.6x to 2x faster than in browsers and Deno? #16798

Open

Labels

api:Javascript

model:transformer

platform:web



josephrocca opened on Jul 21, 2023 · edited by josephrocca

Edits ...

Describe the issue

I'm wondering if the `onnxruntime-web` package has any Node.js-specific optimisations within it when using the `wasm` execution provider? I know there's an `onnxruntime-node` package - but I'm talking specifically about the `onnxruntime-web` package here.

I assumed that since Node.js and Deno and Chrome all use V8, they should have similar performance, but Node.js is about 1.6x faster than the other two.

I'm hoping to be able to achieve the observed Node.js-level performance in Deno and in the browser.

To reproduce

Deno - 510ms

```
// Deno v1.35.2
// deno run --allow-net main.js
await import("https://cdn.jsdelivr.net/npm/onnxruntime-web@1.15.1/dist/ort.js");
ort.env.wasm.wasmPaths = "https://cdn.jsdelivr.net/npm/onnxruntime-web@1.15.1/dist/";
ort.env.wasm.numThreads = 1;
console.log("Downloading model...");
let executionProviders = ["wasm"];
let onnxImageSession = await ort.InferenceSession.create("https://huggingface.co/rocca/op
for(let i = 0; i < 5; i++) {
  let data = new Float32Array(3*224*224).map(n => Math.random()-0.5);
  const feeds = {'input': new ort.Tensor('float32', data, [1,3,224,224])};
  let t = performance.now();
  console.log("Starting inference...");
  const results = await onnxImageSession.run(feeds);
```

```
    console.log(`Finished inference in ${performance.now()-t}ms`);  
  }  
}
```

Chrome Browser - 640ms

Same code as Deno - just paste the above code in your DevTools console.

Node.js - 320ms

```
// Node.js v 18.15.0  
// npm init -y  
// npm install onnxruntime-web@1.15.1  
const ort = require('onnxruntime-web');  
ort.env.wasm.numThreads = 1;  
  
(async function() {  
  console.log("Downloading model...");  
  let executionProviders = ["wasm"];  
  let onnxImageSession = await ort.InferenceSession.create("https://huggingface.co/rocca/  
  for(let i = 0; i < 5; i++) {  
    let data = new Float32Array(3*224*224).map(n => Math.random()-0.5);  
    const feeds = {'input': new ort.Tensor('float32', data, [1,3,224,224])};  
    let t = performance.now();  
    console.log("Starting inference...");  
    const results = await onnxImageSession.run(feeds);  
    console.log(`Finished inference in ${performance.now()-t}ms`);  
  }  
})();
```



Urgency

Not urgent.

Platform

Linux

OS Version

22.04

ONNX Runtime Installation

Released Package

ONNX Runtime Version or Commit ID

1.15.1

ONNX Runtime API

JavaScript

Architecture

X64

Execution Provider

Other / Unknown

Execution Provider Library Version

wasm 1.15.1

Model File

Model used in the above code examples: <https://huggingface.co/rocca/openai-clip-js/resolve/main/clip-image-vit-32-float32.onnx>

Is this a quantized model?

No



github-actions added **api:Javascript** **model:transformer** **platform:web**
on Jul 21, 2023



josephrocca changed the title ~~[Performance] [Web] Using the `onnxruntime-web` package (`wasm` backend) with Node.js is 1.6x faster than in browsers and Deno?~~
[Performance] [Web] Using the `onnxruntime-web` package (`wasm` backend) with Node.js is 1.6x to 2x faster than in browsers and Deno? on Jul 22, 2023



fs-eire on Jul 26, 2023

Contributor



I'm wondering if the onnxruntime-web package has any Node.js-specific optimisations within it when using the wasm execution provider?

What what I know, I don't think there is any.

there are only 2 differences left -

- if SIMD is enabled for all
- `import('https://cdn.jsdelivr.net/npm/onnxruntime-web@1.15.1/dist/ort.js')` comparing to `require('onnxruntime-web')` is something different inside





josephrocca on Jul 30, 2023 · edited by josephrocca

Edits ▼

Contributor

Author



[@fs-eire](#) Sorry for the delayed reply. Can confirm that for the Chrome test, at least, it's using `dist/ort-wasm-simd.wasm`. Not sure how to test on the others. But since all 3 of them are using the same engine (V8), this would seem to indicate that we might be able to get a 2x speedup in browsers (!!) if we can work out what's going on when it's running in the Node.js runtime?

I thought maybe Node.js was ignoring the `wasm.numThreads` option, but that's not the case according to `http` - just a single thread is spiking during computation.

RE your second point, the `node_modules/onnxruntime-web/dist` folder does contain `ort-web.node.js` which IIUC is different to `onnxruntime-web@1.15.1/dist/ort.js`, but I'm guessing this is just because Node.js workers need to be spawned in a different way, and stuff like that - i.e. it's still using V8 (rather than doing FFI stuff to get speed boosts over what V8 can achieve).



guest271314 on Aug 12, 2024



[@josephrocca](#) How did you manage to get `onnxruntime-web` working in `deno`?

I keep getting

```
error: Uncaught (in promise) Error: no available backend found. ERR: [wasm] TypeError:
Invalid URL: '/ort-wasm-simd-threaded.wasm', [cpu] Error: previous call to
'initializeWebAssembly()' failed., [xnnpack] Error: previous call to
'initializeWebAssembly()' failed.
    throw new Error(`no available backend found. ERR: ${errors.map(e => `[${e.name}]
${e.err}`).join(', ')}`);
```



fs-eire on Aug 12, 2024 · edited by fs-eire

Edits ▼

Contributor



It has been quite a while since `onnxruntime v1.15`. I assume that you are using latest version. I am not familiar with Deno, and I assume the error was because it failed to locate the WebAssembly file `ort-wasm-simd-threaded.wasm`. The code uses some ways to try to figure out the path of the current source code, and it seems from the log it is using the root directory. Setting `ort.env.wasm.wasmPaths` may help (depending on how Deno deal with file paths)



guest271314 on Aug 12, 2024



I tried the `env.wasmPaths`. The `./onyx-runtimeweb.js` code is littered with calls to create a Worker, which is preventing me from using the code in Deno and Node.js, et al. Works to an appreciable degree in the browser <https://guest271314.github.io/vits-web/> (<https://github.com/guest271314/vits-web/tree/patch-1>), though is not runtime agnostic.



Add a comment

H B I | ≡ <> 🔗 | ≡ ≡ ≡ | @ ↗ ↶ 📄

Write

Preview

Use Markdown to format your comment

📎 Paste, drop, or click to add files

👍 Close issue

Comment

📄 Remember, contributions to this repository should follow its [contributing guidelines](#), [security policy](#) and [code of conduct](#).

Metadata

Assignees

No one assigned

Labels

api:Javascript model:transformer platform:web

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

 **Subscribe**

You're not receiving notifications from this thread.

Participants

